

An Evolutionary Algorithm for Regression Test Suite Reduction

S.Nachiyappan^{#1} A.Vimaladevi^{#2} C.B.SelvaLakshmi^{#3}

^{#1, #3} Assistant Professor,

*Velammal College Engineering and Technology, Department,
Viraganoor, Madurai*

^{#1}esspee27@gmail.com

^{#3}cbselak08@gmail.com

^{#2}Lecturer, Dept of Applied Science,

*Vivekananda Institute of Engg & Tech. for Women
Tiruchengode.*

^{#2}mca20062009@gmail.com

Abstract— As the software is modified and new test cases are added to the test-suite, the size of the test-suite grows and the cost of regression testing increases. In order to decrease the cost of regression testing, researchers have focused on the use of test-suite reduction techniques, which identify a subset of test cases that provides the same coverage of the software, according to some criterion, as the original test-suite. This paper investigates the use of an evolutionary approach, called genetic algorithms, for test-suite reduction. The proposed model builds the initial population based on test history, calculates the fitness value using coverage and run time of test case, and then selectively breeds the successive generations using genetic operations and allows only the fit tests to the reduced suite. This generational process is repeated until an optimized test-suite is found.

Keywords— Regression testing, coverage, genetic algorithm

I. INTRODUCTION

It is reported that the estimated expenditure of software maintenance exceeds 70% of total software costs and a large percentage of that maintenance is devoted to testing. So it is necessary to enhance the efficiency of software testing. Improvements in the regression testing process would help reduce the cost of software [9].

In this paper we propose a mathematical model to reduce the cost of regression testing by minimizing the size of the test-suite using an evolutionary algorithm called Genetic Algorithm. Generally a limited time is allotted for testing. And hence those test cases which violate the time constraint and coverage value will be declared as rejected. We make use of mutation and cross-over operations of Genetic Algorithm in order to achieve an optimal sized test-suite.

Regression testing is an expensive testing process to validate any modified software and detect whether new faults have been introduced into previously tested code, which is a

great deal. In one example, an industrial collaborator has reported that for one of its products of approximately 20,000 lines of code (LOC) requires seven weeks to run the entire test-suite [3].

II. RELATED WORKS

Genetic algorithm can be implemented to reduce the cost of regression testing. This methodology uses a mathematical model to reduce the test-suite by transforming it into a linear integer programming form. The GA is the combination of block-based coverage and test execution cost. Hence the reduction will be obtained up to 89% [9]. In another case; genetic algorithms were used to search for sorting routines. The result shows that best sorting routines are obtained when using genetic algorithms. The generated sorting algorithm is on average 36% faster than the best pure sorting routines on seven different platforms on which they achieved up to 42% faster algorithm [2].

Another methodology is one in which the unnecessary test cases are removed. A redundant test case is one whose removal does not affect the testing coverage. A representative set is selected and it can be used to reduce the size of the test-suite by replacing the redundant test cases by this representative set. We can achieve 40% of test case reduction in this methodology [5].

Some of the empirical studies on selecting cost-effective test case prioritization shows the prioritization techniques (Random ordering, Optimal ordering etc.,) vary considerably across various attributes of the program, test-suites and modifications being considered. Due to this variation it is difficult for a tester to choose an appropriate prioritization technique. Considering the conditions under which they are or are not suitable, we have to choose appropriate prioritization technique [1, 3].

Test case prioritization techniques organize the test cases in a test suite, allowing for an increase in the effectiveness of testing. Prioritization technique using a genetic algorithm (GA) prioritizes subsequences of the original test suite so that the new suite, which is run within a time-constrained execution environment, will have a superior rate of fault detection when compared to rates of natively prioritized test suites [9].

A methodology to control the size of test-suite for modified condition/ decision coverage is an effective verification technique, used to eliminate the redundant test cases and select essential test cases for reduced set. The minimization of test-set performed in this way does not affect the fault finding capabilities of the test-suites.

Genetic algorithm is used for prioritizing the test-suite in light of time constraint. Here reordering of test-suite increase the fault detection rate. The fault detection rate is based on coverage information. This method had up to 120% of improvement in APFD (Average Percentage of Fault Detected) over the other prioritizations [4].

III. PROPOSED MODEL

In this paper we propose a new mathematical model to reduce the cost of regression testing by minimizing the size of the test-suite using an evolutionary algorithm called Genetic Algorithm. Based on the Block-based coverage value and the execution time of the test cases the fitness value will be calculated. The test cases are selected if it has an optimum fitness. Since a limited time is allotted for testing the test case which violates the time constraint will be rejected. In some cases they may hopefully give a good coverage. And hence we apply genetic algorithm for those rejected test cases, thereby achieving an optimal sized test-suite. In order to promote those test cases we make use of mutation and cross-over operations of GA.

In this paper we propose an adaptive cross-over technique in which n-point cross-over can be applied. The parents are selected from the rejected pool. Then cross-over is applied to those individuals. The resulting test cases are promoted to the new generation, only if it satisfies the time constraint. If the parents are consecutive then we cannot apply cross-over since it will not produce any fit individuals out of it. The alternate solution for cross-over is mutation. Here, we propose an adaptive mutation technique where n-point mutation is applied to the rejected individuals. This operation mutates the specified bit only if it satisfies the mutation criterion.

IV. PROPOSED GENETIC ALGORITHM

Input: Test-suite T

Coverage c_i

Run-time r_i

Mutation probability pm

Cross-over probability pc

Output: Reduced Test-suite T'

Begin:

Read the test cases, their run time and coverage information.

Calculate the fitness value for each individual

Loop:

Select the individuals for reproduction (t_i and t_j)

If random < pc then perform crossover

Else perform mutation

Repeat loop until optimized suite is obtained.

End

V. TEST CRITERIA

There are a lots of criterion with which we can find the fitness function of the individual test cases. Some of them are fault detection rate (APFD) - *Average Percentage of Faults Detected*, - APFD metric was used to determine the effectiveness of the new test case orderings, cost, coverage information, run-time etc., the main aim of our paper is to minimize the test cases by calculating the fitness function. In real time we will find the fitness of a person, either by health or wealth, similarly there are many test criteria's available to find the fitness calculation. Here we are focusing Cost and Coverage because Cost plays a vital role in a project. We choose this approach for reducing the cost and focusing maximum block coverage. Since they seem to highly influence the fitness of test case when compared with the rest.

VI. EXECUTION TIME

It is the amount of time taken by the test case to execute. If the execution time is low for the test case then it implies that it is best-fit. Since time is important factor is determining the fitness, we make use of run time of each test case in finding their fitness function. Since a limited time is allotted for testing during the software development process. The execution time can be recorded using the testing tools. Here we used quick test professional for finding the execution time of individual test case.

VII. TEST COVERAGE

The metrics used to measure testing thoroughness include statement testing (whether each statement in the program has been executed at least once), branch testing (whether each exit from each branch has been executed at least once) and path testing (whether all logical paths, which may involve repeated execution of various segments, have been executed at least once). Statement testing is the coverage metric most frequently used as it is relatively simple to implement. the conference paper formatting requirements is to use this document as a template and simply type your text into it.

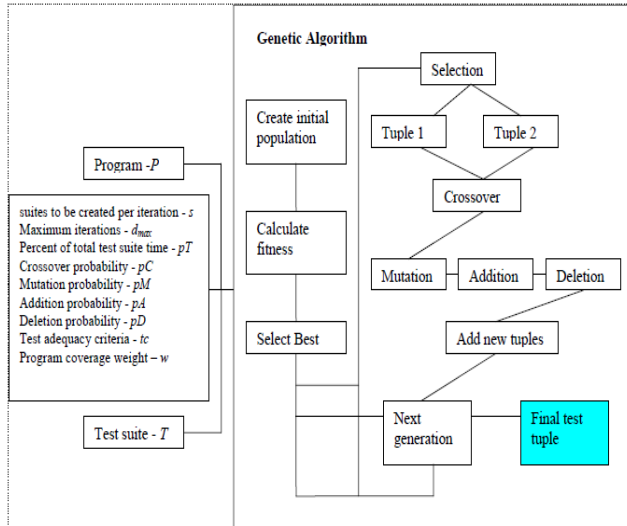
Apart from the above mentioned metrics two forms of test adequacy criteria is considered. (i) method coverage: a method is covered when it has been entered, (ii) block coverage: a basic block is a sequence of statements or instructions without any jump targets is covered whenever it is entered because several high level languages source statements can be in the same basic block, it is sensible to keep track of basic blocks rather than individual statements

and time of execution. Our genetic algorithm accepts the coverage information based on the code covered in an application by an entire test-suite.

The fitness value for each individual is calculated using the coverage and its cost. The fitness function for a test case t_i can be compared as follows.

$$F(T_i) \propto \frac{\text{coverage } i}{\text{time } t} \quad (1)$$

$$F(T_i) = k \frac{\text{coverage } i}{\text{time } t} \quad (2)$$



If the test case is having a good fitness then there is more possibility the case can be selected. Because the higher fitness value denotes that the test case covers more blocks with less cost.

TABLE 1. FINDING FITNESS FUNCTION FOR INDIVIDUALS.

Test case	Coverage	Runtime	Fitness
t1	0.2	5	0.04
t2	0.6	6	0.10
t3	0.4	1	0.40
t4	0.2	3	0.06
t5	0.4	4	0.10
t6	0.6	5	0.12
t7	0.2	9	0.02
t8	0.1	2	0.05

Minimization using Genetic Algorithm

Genetic Algorithm is one form of evolutionary algorithm often used for optimization problems.

VIII. SELECTION

The fit individuals are selected for either cross over or mutation using selection process. In this work we used roulette wheel selection algorithm. Here the individuals are selected with a probability proportional to their fitness. After the fitness values are assigned to each individual, they are normalized in relation to the rest of the population by multiplying each value by a constant. Then, the population is sorted by descending fitness value and accumulated normalized fitness values are calculated.

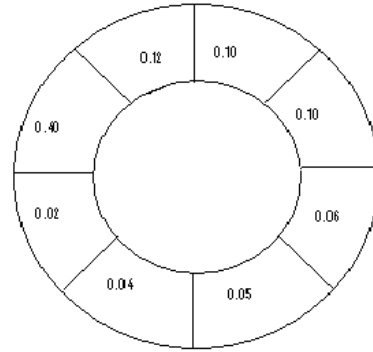


Fig 1. Roulette wheel selection

in two column format with a space of 4.22mm (0.17) between columns.

IX. GENETIC OPERATORS

In this paper we use the genetic operators such as cross-over and mutation to derive the new off springs to introduce changes in the existing test-suite. Mutation operator operates on an individual whereas cross-over operates on a pair of individuals.

X. CROSS-OVER

The method of merging the information units of two individuals, if coding is chosen properly. The good parents produce good children. Here we are cutting the two strings at the user provided cross over point and swapping the two fails. The outcome of this process is stored as new population.

BEFORE CROSS OVER AFTER CROSS OVER

101001	11	10100100
110011	00	11001111

2-point cross-over

XI. MUTATION

Mutation is the process of deformation of the strip with a certain probability. In this work, we performed adaptive mutation & hence the mutation is carried out at the first specified by the user. The mutation can be done only if the random value is less than the mutation probability & it is specified by the user.

for t_1 to t_n

do
if Random [0,1] < P_m then
inverts[j].

where j – mutation point specified by the user.

be indented. All paragraphs must be justified, i.e. both left-justified and right-justified.

XII. TERMINATION

The Generally the average degree of fitness will have increased by this procedure for the reduced test-suite pool, since mainly the best organisms from the first generation are selected for breeding. The algorithm is repeated until an organism is produced which gives a solution that is good enough. The terminating condition usually consists of either satisfying a problem-specific success predicate or completing a specified number of generations. For our concern we assumed the terminating condition as the point where we obtain an optimal sized test suite. As testing is a never ending process we can never assume that 100 % testing has been done, we can only minimize the risk of shipping the product to client with X testing done.

- 1) EMPIRICAL STUDIES
- 2) EXPERIMENTATION

Experimentation is done as two categories. The first category examines the efficiency of proposed algorithm for a student project and the second category tests two industrial projects.

CATEGORY-I

To investigate the effectiveness of our proposed genetic algorithm, we considered a student project and derived test cases for that, for ease of understanding all the test cases were data driven test cases and we found the number of blocks covered by each test case. Using that information we calculated the percentage of coverage using the formula, where BC is the block coverage.

% of BC=no. of blocks covered/total no. of blocks

XIII. RESULT ANALYSIS

The results obtained after obtaining a minimized test set using our proposed model shows a good optimal sized test set and it is a highly adaptive method. The chart shown below depicts the percentage of code covered by the original test cases can cover the same block. We are reducing the size by selecting the fit test cases by applying GA. Experimentation is performed to check the effectiveness of our GA. We can infer from the above chart that the reduction does not show much impact on coverage.

TABLE 2. BLOCK COVERAGE PERCENTAGES OF INDIVIDUAL TESTS

Test case	B1	B2	B3	B4	B5
t1				#	
t2	#	#	#		
t3	#			#	
t4					#
t5	#	#	#		
t6					#
t7					#
t8	#	#	#		

TABLE 3. BLOCK COVERAGE RATIO OF REDUCED SUITE

Test case	B1	B2	B3	B4	B5
t1				#	
t3	#			#	
t4					#
t5	#	#	#		
t6					#
t8	#	#	#		

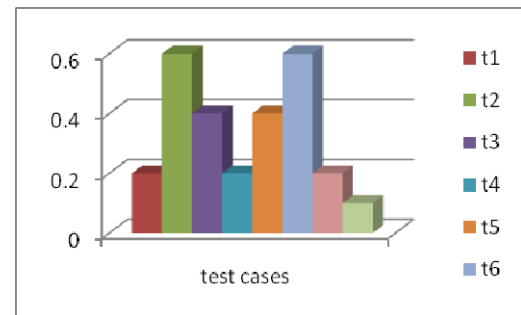


Fig 2. Coverage of original suite (y-axis coverage & x-axis test cases)

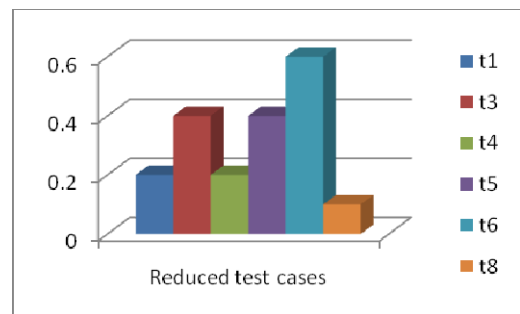


Fig 3. Coverage of reduced suite (y-axis coverage & x-axis test cases)

CATEGORY-II

Two industrial case studies are analyzed by conducting a post hoc analysis of the test efforts at Cosmo soft Technologies limited, India, a leading developer of ERP projects and Tech Zone, a leading developer of JAVA projects

in India. The first case study (Cosmo soft Technologies) involves analyzing employee self service portal (ESSP) written in JAVA language. It comprises of 73,525 changed lines of code, built upon approximately a 300,000 lines of code base; 25 modified requirements, and 100 system level test cases. The second case study (Tech Zone) involve in analyzing BSNL token sending program (BTSP), written in JAVA, comprises of 100,000 lines of code, 20 new requirements and 100 system level test cases.

The following steps are performed for the two industrial case studies.

1. Computation of block coverage and method coverage using EMMA tool.
2. Minimize the test cases based on block coverage according to the proposed minimization technique.
3. Minimize the test cases based on method coverage according to the proposed minimization technique.
4. Execute the test cases in proposed minimization order, original order, block coverage order and method coverage order.

The coverage values for 20%, 40%, 60%, 80%, 100% of execution is compared and the graph is plotted with the percentage of coverage on X-axis and corresponding number of executed test cases on Y-axis for both the projects. The test results are then analyzed considering if the test cases are run in original order, minimization based on block coverage order and minimization function coverage order.

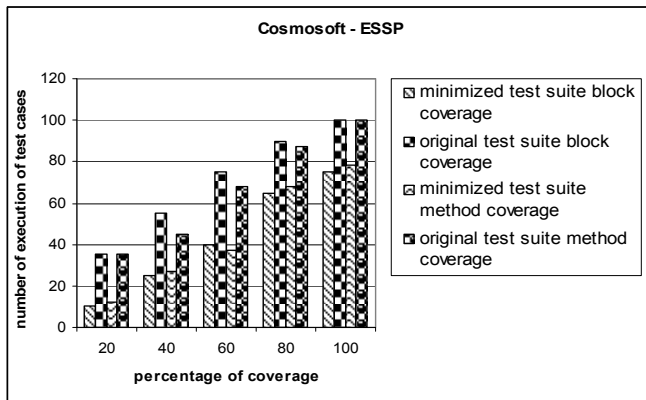


Fig 4 Percentage of coverage

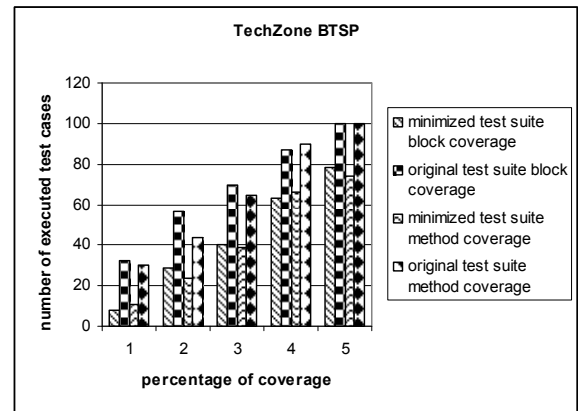


Fig 5. Percentage of coverage

XIV. DISCUSSION AND CONCLUSION

In this we have presented a novel model for test set reduction problem by using an evolutionary algorithm called Genetic Algorithm. Our GA is capable of producing an optimal sized test-suite taking execution time and coverage factors into account that can be inferred from the experimentation and analysis section. The results shows that coverage is not compensated in the reduced test set. Also, the reduced test-suite has the test cases that cover all the blocks of the program under test at least once. From the experiments analysis we conclude that this method of reduction has the potential to provide an effectively minimized test cases for applications. Increased granularity of fitness function led to more effective test case reduction as well.

XV. FUTURE SCOPE

There are many directions that future on this project could take. First of all, the scalability of GA should be considered. If we can calculate other coverage values like statement coverage, decision coverage etc., a different method can be used to calculate the fitness as we did. Next more experimentation should be completed in an effort to improve the genetic algorithm. In this work, only the values of optimal test set size, time constraint, and the type of coverage are considered, there are numerous way to perform cross over and mutation that could prove to be more effective than the current method. Due to the importance of test case execution time in relation to coverage in this work, we can also consider execution time in relation with fault finding using APFD metric.

REFERENCES

- [1] S. Elbaum, G. Rothermel, Satya Kanduri, A. G. Malishevsky, *Selecting a cost-effective test case prioritization technique* software quality journal, vol.12, no. 3, 2004
- [2] Xiaoming Li, Maria Jesus Garzaran and David Padua, *Optimizing sorting with genetic algorithm*, In proceedings of the international symposium on Code Generation and Optimization (CGO'05).
- [3] S. Elbaum, A. G. Malishevsky and G. Rothermel. *Test case prioritization: A family of empirical studies*. IEEE transactions on software engineering, vol 28, no: 2, pages 159-182, Feb-2002.
- [4] K. R .Walcott, M. L. Soffa, G. M. Kapfhammer and R. S. Roos. *Time aware test-suite prioritization* ISSA'06, July 17–20, 2006, Portland, Maine, USA.

- [5] M. J. Harrold, R. Gupta and M. L. Soffa, *A methodology for controlling the size of a test-suite*, In conference on software maintenance-1990, pages 302-310, San Diego, CA, Nov 1990.
- [6] J. Offutt, J. Pan and J. M. Voas, *Procedures for reducing the size of coverage-based test-suites*, in proceedings of 12th international conference on testing computer software, pages 111-123, June 1995.
- [7] Xue-ying MA, Zhen-feng HE, Bin-kui SHENG and Cheng-qing YE, *A Genetic Algorithm for Test-Suite Reduction*, Advance Parallel Processing Techniques, 6th international workshop (APPT), pages 253-262, Oct 2005, Hong Kong, China.
- [8] WWW: <http://www.flll.jku.at>
- [9] www.cs.virginia.edu/~krw7c/GAprioritizations/
- [10] Genetic Algorithms: Theory and Applications, lecture notes, third edition-winter 2003-2004 by Ulrich Bodenhofer.
- [11] www.SofTReL.org.